Cours PHP

Objectifs :

- savoir écrire une page PHP dynamique,
- traiter et valider un formulaire,
- conserver un utilisateur connecté avec une session,
- interroger une base MariaDB avec PDO,
- appliquer les règles de sécurité de base (XSS, injection SQL, mots de passe).

Prérequis : HTML/CSS de base, notions d'algorithmique (variables, conditions, boucles), principe du modèle client/serveur (HTTP).

Ressources associées : exercices dans `02_Exercices/Exercices_PHP.pdf` · TP1 Carnet de contacts et TP2 Boutique dans `03_TP/` · approfondissement des sessions et cookies dans `01_Cours/Cours_Sessions_Cookies.pdf`.

---

1. Introduction à PHP

- **Langage côté serveur** : le serveur exécute le code PHP et envoie uniquement le **HTML généré** au navigateur.
- Permet de créer des **pages dynamiques** (contenu qui dépend de l'utilisateur, de l'heure, d'une base de données…).
- Nécessite : un **serveur web** (Apache, Nginx), **PHP**, et éventuellement une **base de données** (MySQL/MariaDB).
- Les fichiers PHP ont l'extension `.php`.



Rappel : le navigateur ne voit jamais le code PHP, seulement le HTML généré. Chaque requête est **indépendante** : le script s'exécute, produit sa réponse, puis se termine (les variables PHP sont perdues — voir les sessions, section 11).

---

2. Installation et environnement

En local avec XAMPP

- XAMPP = Apache + MariaDB + PHP + Perl.
- Téléchargez-le sur : apachefriends.org

- Installation par défaut, puis lancer **Apache** et **MySQL** (le module affiché « MySQL » démarre MariaDB) depuis le panneau de contrôle.
- Placer vos fichiers PHP dans le dossier `htdocs`.

Exemple :

```
htdocs/
    cours_TP_PHP/
        index.php
```

Accès : `http://localhost/cours_TP_PHP/`

Remarques :

- Un fichier qui contient du PHP doit être traité par PHP : il porte l'extension `.php` **et** doit être servi par un serveur configuré pour PHP. Renommer un fichier en `.php` ne suffit pas à activer PHP.
- Ouvrir le fichier directement (`file:///C:/xampp/htdocs/...`) n'exécute **pas** le PHP : passez toujours par `http://localhost/...`.
- Si Apache ne démarre pas, le port 80 est probablement déjà utilisé (autre serveur web, Skype, IIS…).

Alternative : serveur intégré de PHP

Si PHP est installé en ligne de commande, il suffit de se placer dans le dossier du projet :

```
php -S localhost:8000
```

Accès : `http://localhost:8000/`. Pratique pour tester vite, mais sans MariaDB ni Apache.

Vérifier l'installation

Créer `info.php` :

```php
<?php
phpinfo();
```

Elle affiche la version de PHP, les extensions chargées (`pdo_mysql`, `mbstring` …) et la configuration. **Supprimer ce fichier** après usage : il révèle des informations sensibles sur le serveur.

---

3. Syntaxe de base

```php
<?php
echo "Bonjour !";
?>
```

- Début : `<?php` / Fin : `?>`
- Les instructions se terminent par `;`
- PHP et HTML peuvent être mélangés dans un même fichier.
- Dans un fichier qui ne contient **que** du PHP, on omet la balise fermante `?>` (elle évite d'envoyer par erreur des espaces ou des retours à la ligne après le code).

- Raccourci d'affichage dans les pages HTML : `<?= $variable ?>` équivaut à `<?php echo $variable; ?>`.

Commentaires

```
// commentaire sur une ligne
# autre commentaire sur une ligne
/* commentaire
   sur plusieurs lignes */
```

Afficher du texte

`echo` est l'instruction d'affichage la plus courante (`print` existe aussi, mais `echo` est préférée). Ce ne sont pas des fonctions : les parenthèses sont facultatives.

```
$txt1 = "PHP";
$txt2 = "Informatique et réseaux";
$x = 5;
$y = 4;

echo "Affichage :";
echo "<h2>" . $txt1 . "</h2>";
echo "Etudier le PHP en " . $txt2 . "<br>";
echo $x + $y;
```

Code HTML envoyé au navigateur :

```
Affichage :<h2>PHP</h2>Etudier le PHP en Informatique et réseaux<br>9
```

Rendu dans le navigateur : le texte « Affichage : », puis le titre **PHP** (balise `<h2>`), puis « Etudier le PHP en Informatique et réseaux » et, à la ligne, `9`.

> Affichage :
>
> ## PHP
>
> Etudier le PHP en Informatique et réseaux
> 9

Déboguer : `var_dump()` et `print_r()`

`var_dump()` affiche la valeur d'une variable, **son type et sa structure** (tableaux, objets). C'est l'outil de débogage de base.

```
$nom = "Carnus";
$age = 20;
$rgb = array("rouge", "vert", "bleu");
$c = [1.5, 2.3, 5];
var_dump($nom);
var_dump($age);
var_dump($rgb);
var_dump($c);
```

Résultat :

```
string(6) "Carnus"
int(20)
array(3) {
  [0]=> string(5) "rouge"
  [1]=> string(4) "vert"
  [2]=> string(4) "bleu"
}
array(3) {
  [0]=> float(1.5)
  [1]=> float(2.3)
  [2]=> int(5)
}
```

> Dans un navigateur, les retours à la ligne sont ignorés : entourer l'appel de `<pre>` … `</pre>` pour obtenir cet affichage lisible. `print_r($tableau)` donne un affichage plus compact (sans les types).

---

4. Variables, constantes et types

Variables

- Commencent par `$`, suivi d'une lettre ou de `_` (pas d'un chiffre).
- Sensibles à la casse (`$age` ≠ `$Age`).
- Créées lors de la première affectation (pas de déclaration de type obligatoire).

```php
$nom = "Carnus";
$age = 20;
echo "$nom a $age ans";
```

Guillemets doubles ou simples ?

```php
$nom = "Carnus";
$tab = ["p" => "Charles"];
echo "Bonjour $nom";          // Bonjour Carnus   (la variable est remplacée)
echo 'Bonjour $nom';          // Bonjour $nom     (texte brut)
echo "Prénom : {$tab['p']}"; // Prénom : Charles (accolades pour les tableaux)
```

Constantes

Une constante ne change pas pendant l'exécution (pas de `$`).

```php
const TVA = 0.2;
define("NOM_SITE", "Boutique");
echo TVA, " ", NOM_SITE;
```

Types principaux

Type	Exemple
String	`$txt = "Bonjour";`
Integer	`$x = 10;`
Float	`$pi = 3.14;`
Boolean	`$ok = true;`
Array	`$tab = ["rouge","bleu"];`
Object	`new MaClasse();`
NULL	`$x = null;`

Outils sur les types :

```php
echo gettype(3.14);        // double
var_dump(is_numeric("12")); // bool(true)
$n = (int) "42abc";         // conversion (cast) : 42
```

Typage dynamique : attention aux comparaisons

PHP convertit automatiquement les types pour comparer (`==`). Cela peut surprendre :

```php
var_dump("1" == "01");   // true  (comparaison de nombres)
var_dump(0 == "a");      // false (PHP 8)
var_dump("0" == false);  // true
```

> **Règle** : utiliser `===` et `!==` (comparaison de la valeur **et** du type) dès que possible.

Opérateurs arithmétiques

Opérateur	Nom	Syntaxe
`+`	Addition	`$x + $y`
`-`	Soustraction	`$x - $y`
`*`	Multiplication	`$x * $y`
`/`	Division	`$x / $y`
`%`	Modulo (reste)	`$x % $y`
`**`	Exponentiation	`$x ** $y`

`intdiv($x, $y)` donne le quotient entier. Raccourcis : `$x += 2;`, `$x -= 2;`, `$x++;`, `$x--;`.

> En PHP 8, une division ou un modulo par **zéro** lève une erreur (`DivisionByZeroError`) : il faut tester le diviseur avant de calculer.

Fonctions mathématiques

```php
echo pi();                      // 3.1415926535898
echo min(0, 150, 30, 20, -8, -200); // -200
echo max(0, 150, 30, 20, -8, -200); // 150
echo abs(-6.7);                 // 6.7
echo sqrt(64);                  // 8
echo round(0.60);               // 1
echo round(0.49);               // 0
echo round(3.14159, 2);         // 3.14
echo rand(1, 6);                // entier aléatoire entre 1 et 6
```

Opérateurs de comparaison

Opérateur	Nom	Exemple	Résultat
`==`	Égal	`$x == $y`	`true` si `$x` est égal à `$y`
`===`	Identique	`$x === $y`	`true` si `$x` est égal à `$y` **et** du même type
`!=`	Différent	`$x != $y`	`true` si `$x` est différent de `$y`
`<>`	Différent	`$x <> $y`	`true` si `$x` est différent de `$y`
`!==`	Non identique	`$x !== $y`	`true` si `$x` est différent de `$y`, ou n'est pas du même type
`>`	Supérieur	`$x > $y`	`true` si `$x` est supérieur à `$y`
`<`	Inférieur	`$x < $y`	`true` si `$x` est inférieur à `$y`
`>=`	Supérieur ou égal	`$x >= $y`	`true` si `$x` est supérieur ou égal à `$y`
`<=`	Inférieur ou égal	`$x <= $y`	`true` si `$x` est inférieur ou égal à `$y`

Opérateurs logiques

Opérateur	Nom	Exemple	Résultat
`and`	Et	`$x and $y`	`true` si `$x` et `$y` sont vrais
`&&`	Et	`$x && $y`	`true` si `$x` et `$y` sont vrais
`or`	Ou	`$x or $y`	`true` si `$x` ou `$y` est vrai
`||`	Ou	`$x || $y`	`true` si `$x` ou `$y` est vrai
`!`	Non	`!$x`	`true` si `$x` est faux

> `and` / `or` ont une priorité **plus faible** que `=` : `$a = true and false;` affecte `true` à `$a`. Préférer `&&` et `||`.

Opérateurs de chaîne

Opérateur	Nom	Exemple	Résultat
`.`	Concaténation	`$txt1 . $txt2`	Concaténation de `$txt1` et `$txt2`
`.=`	Affectation avec concaténation	`$txt1 .= $txt2`	Ajoute `$txt2` à `$txt1`

Opérateur de coalescence `??`

Renvoie la valeur de gauche si elle **existe et n'est pas** `null`, sinon la valeur de droite. Indispensable pour lire `$_GET` / `$_POST` sans erreur :

```php
$prenom = $_POST["prenom"] ?? "";   // "" si le champ n'a pas été envoyé
```

---

5. Structures de contrôle

Conditions

```php
$x = 3;

if ($x > 0) {
    echo "Positif";
} elseif ($x < 0) {
    echo "Négatif";
} else {
    echo "Zéro";
}
```

`switch` : compare une valeur à plusieurs cas. Sans `break`, l'exécution continue dans le cas suivant.

```php
$x = 1;
switch ($x) {
    case 0:
        echo '$x = 0';
        break;
    case 1:
        echo '$x = 1';
        break;
    default:
        echo '$x contient une autre valeur';
}
```

```php
$jour = (int) date("N");   // 1 = lundi ... 7 = dimanche
switch ($jour) {
    case 6:
        echo "Samedi";
        break;
    case 7:
        echo "Dimanche";
        break;
    default:
        echo "Jour de semaine";
}
```

`match` (PHP 8) : version plus concise qui **renvoie une valeur** et compare avec `===`.

```php
$libelle = match ($jour) {
    6, 7    => "Week-end",
    default => "Semaine",
};
echo $libelle;
```

Boucles

```php
for ($i = 0; $i < 5; $i++) {
    echo $i . "<br>";
}

$couleurs = ["rouge", "vert"];
foreach ($couleurs as $c) {
    echo $c . "<br>";
}

$x = 1;
while ($x <= 4) {
    echo "Le nombre est : $x <br>";
    $x++;
}

$y = 1;
do {
    echo "Le nombre est : $y <br>";
    $y++;
} while ($y <= 4);
```

- `break` : sort de la boucle. `continue` : passe à l'itération suivante.
- `do … while` s'exécute **au moins une fois** (la condition est testée à la fin).
- Attention à la condition d'arrêt : une boucle `while` dont la condition reste vraie ne se termine jamais.

---

6. Fonctions

```php
function addition(float $a, float $b = 0): float {
    return $a + $b;
}
echo addition(5, 3); // 8
```

- **Paramètres par défaut** : `$b = 0`
- **Types déclarés** : `float $a` (type du paramètre) et `: float` (type du retour). Ils documentent le code et détectent des erreurs.
- **Retour** : `return` (la fonction s'arrête à cet endroit).
- **Passage par référence** : `&` permet de modifier la variable d'origine.

```php
function inc(int &$x): void {
    $x++;
}
$n = 5;
inc($n);
echo $n; // 6
```

Portée des variables

Une variable créée dans une fonction est **locale** : elle n'existe pas en dehors, et les variables extérieures ne sont pas visibles à l'intérieur.

```php
$tva = 0.2;

function prixTtc(float $ht, float $tva): float {
    return $ht * (1 + $tva);   // $tva est reçue en paramètre
}
echo prixTtc(100, $tva); // 120
```

> Éviter `global` : passer les valeurs en paramètres rend les fonctions réutilisables et testables.

---

7. Tableaux

Tableaux indexés

```php
$marques = ["HP", "Apple", "Dell"];
echo $marques[0];          // HP
$marques[] = "Lenovo";     // ajoute à la fin
echo count($marques);      // 4
```

Tableaux associatifs

```php
$ages = ["Charles" => 19, "Carnus" => 20];
echo $ages["Carnus"]; // 20
$ages["Anna"] = 18; // ajout
```

Boucle sur un tableau associatif :

```php
foreach ($ages as $nom => $age) {
    echo "$nom a $age ans<br>";
}
```

Tableaux à plusieurs dimensions

Un tableau peut contenir des tableaux (c'est la structure utilisée dans le TP1 Carnet de contacts) :

```php
$contacts = [
    ["nom" => "Charles", "email" => "charles@carnus.fr"],
    ["nom" => "Carnus", "email" => "carnus@carnus.fr"],
];
echo $contacts[1]["nom"]; // Carnus

foreach ($contacts as $contact) {
    echo $contact["nom"] . " : " . $contact["email"] . "<br>";
}
```

Fonctions utiles

Fonction	Rôle
`count($t)`	Nombre d'éléments
`in_array($v, $t)`	`true` si la valeur est dans le tableau
`array_key_exists($k, $t)`	`true` si la clé existe
`sort($t)` / `rsort($t)`	Tri croissant / décroissant (réindexe)
`ksort($t)` / `asort($t)`	Tri d'un tableau associatif par clé / par valeur
`array_sum($t)`	Somme des valeurs
`implode(", ", $t)`	Tableau → chaîne
`explode(",", $chaine)`	Chaîne → tableau
`unset($t[1])`	Supprime un élément

7.1 — Les variables superglobales

PHP met à disposition plusieurs variables particulières appelées **variables superglobales** (*superglobals*).

Ce sont des variables prédéfinies qui sont accessibles depuis n'importe quelle partie du script PHP.

Elles permettent de récupérer :

- Les données transmises par un formulaire.
- Les paramètres présents dans l'URL.
- Les informations concernant le serveur et la requête HTTP.
- Les fichiers envoyés par un utilisateur.
- Les données stockées dans une session.
- Les cookies enregistrés par le navigateur.

Les principales variables superglobales sont :

Variable	Rôle
`$_GET`	Récupérer les paramètres transmis dans l'URL.
`$_POST`	Récupérer les données transmises par un formulaire en POST.
`$_SERVER`	Obtenir des informations sur le serveur et la requête HTTP.
`$_FILES`	Accéder aux informations concernant les fichiers téléversés.
`$_SESSION`	Accéder aux données de session.
`$_COOKIE`	Accéder aux cookies envoyés par le navigateur.
`$_REQUEST`	Regrouper certaines données issues de GET, POST et COOKIE.
`$_ENV`	Accéder aux variables d'environnement.
`$GLOBALS`	Accéder aux variables globales du script.

Remarque : les superglobales sont généralement des tableaux associatifs. On accède à leurs éléments à l'aide de clés, comme pour les tableaux classiques.

La superglobale `$_GET`

La variable `$_GET` permet de récupérer les paramètres transmis dans l'URL d'une page. Ces paramètres apparaissent après le caractère `?` et sont séparés par le caractère `&`.

Exemple - Considérons l'URL suivante :

```
http://localhost/index.php?nom=Dupont&age=25
```

PHP permet de récupérer les valeurs de ces paramètres :

```php
<?php
echo $_GET['nom']; // Dupont
echo $_GET['age']; // 25
?>
```

Chaque paramètre de l'URL devient une entrée du tableau associatif `$_GET`.

Exemple avec vérification - Il est préférable de vérifier qu'un paramètre existe avant de l'utiliser :

```php
<?php
if (isset($_GET['nom'])) {
    echo "Bonjour " . htmlspecialchars($_GET['nom']);
} else {
    echo "Aucun nom n'a été transmis.";
}
?>
```

La fonction `isset()` permet de vérifier qu'une variable ou une entrée existe et n'est pas égale à `null`.

Remarque : les paramètres GET sont visibles dans l'URL. Il ne faut donc jamais transmettre de mots de passe ou d'informations confidentielles de cette manière.

La superglobale `$_POST`

La variable `$_POST` permet de récupérer les données transmises par un formulaire HTML utilisant la méthode POST.

Contrairement à GET, les données ne sont pas directement ajoutées à l'URL.

Exemple de formulaire HTML

```
<form action="traitement.php" method="POST">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom">

    <label for="email">Adresse e-mail :</label>
    <input type="email" id="email" name="email">

    <button type="submit">Envoyer</button>
</form>
```

Récupération des données en PHP - Dans le fichier `traitement.php` :

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $nom = $_POST['nom'] ?? '';
    $email = $_POST['email'] ?? '';

    echo "Nom : " . htmlspecialchars($nom) . "<br>";
    echo "E-mail : " . htmlspecialchars($email);

}
?>
```

L'opérateur `??` permet de fournir une valeur par défaut lorsque la variable ou la clé n'existe pas ou vaut `null`.

La superglobale `$_SERVER`

La variable `$_SERVER` contient des informations concernant le serveur web et la requête HTTP en cours.

Elle permet de connaître :

- Le nom du serveur.
- Le nom du fichier PHP exécuté.
- La méthode HTTP utilisée.
- Le chemin de la requête.
- Certaines informations concernant le navigateur.

Exemple

```
<?php
echo $_SERVER['SERVER_NAME'];
echo $_SERVER['PHP_SELF'];
echo $_SERVER['REQUEST_METHOD'];
echo $_SERVER['REQUEST_URI'];
?>
```

Clé	Description
`SERVER_NAME`	Nom du serveur.
`PHP_SELF`	Chemin du script PHP courant.
`REQUEST_METHOD`	Méthode HTTP utilisée (GET, POST...).
`REQUEST_URI`	URI demandée par le navigateur.
`HTTP_USER_AGENT`	Informations communiquées par le navigateur.
`REMOTE_ADDR`	Adresse IP du client, telle qu'observée par le serveur ou son environnement réseau.
`SCRIPT_NAME`	Chemin du script exécuté.

Exemple : distinguer GET et POST

```php
<?php

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    echo "La page a été demandée avec GET.";
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    echo "Le formulaire a été envoyé avec POST.";
}

?>
```

---

8. Chaînes de caractères et expressions régulières

Fonctions courantes

Fonction	Rôle	Exemple → résultat
`trim($s)`	Supprime les espaces en début et fin	`trim(" ab ")` → `"ab"`
`mb_strlen($s)`	Nombre de **caractères**	`mb_strlen("été")` → `3`
`strlen($s)`	Nombre d'**octets** (≠ caractères avec les accents)	`strlen("été")` → `5`
`mb_strtoupper($s)`	Majuscules (accents compris)	`mb_strtoupper("été")` → `"ÉTÉ"`
`str_contains($s, $sous)`	`true` si `$s` contient `$sous` (PHP 8)	`str_contains("abc", "b")` → `true`
`str_replace($a, $b, $s)`	Remplace `$a` par `$b`	`str_replace("a", "o", "dada")` → `"dodo"`
`substr($s, $debut, $long)`	Extrait une partie de la chaîne	`substr("bonjour", 0, 3)` → `"bon"`
`sprintf("%.2f", $x)`	Formate un nombre	`sprintf("%.2f", 3.14159)` → `"3.14"`
`number_format($x, 2, ',', ' ')`	Formate pour l'affichage	`number_format(1234.5, 2, ',', ' ')` → `1 234,50`

> Pour compter les caractères d'un mot de passe ou d'un nom saisi par un utilisateur, utiliser `mb_strlen()`.

Expressions régulières avec `preg_match()`

Une expression régulière (« regex ») décrit un motif de texte. `preg_match($motif, $chaine)` renvoie `1` si la chaîne correspond, `0` sinon.

```php
// Identifiant : 3 à 20 caractères, lettres, chiffres ou underscore
$motif = '/^[A-Za-z0-9_]{3,20}$/';
var_dump(preg_match($motif, "etudiant_1")); // int(1)
var_dump(preg_match($motif, "a b"));        // int(0)
```

Élément	Signification
`^` … `$`	Début et fin de la chaîne (motif sur *toute* la chaîne)
`[A-Za-z]`	Un caractère parmi cet ensemble
`\d`	Un chiffre
`{3,20}`	Entre 3 et 20 répétitions
`+` / `*`	1 ou plusieurs / 0 ou plusieurs
`(?=.*\d)`	« Contient au moins un chiffre » (assertion)

Exemple de règle de mot de passe (8 caractères minimum, au moins une majuscule et un chiffre) :

```php
$motif = '/^(?=.*[A-Z])(?=.*\d).{8,}$/';
var_dump(preg_match($motif, "Abcdefg1")); // int(1)
var_dump(preg_match($motif, "abcdefgh")); // int(0)
```

---

9. Formulaires et validation – Méthodes GET et POST

Différences entre GET et POST

Méthode	Caractéristiques	Usage
GET	Les données sont envoyées dans l'URL (`?nom=Carnus&age=20`) — visibles, mises en favori, conservées dans l'historique et les journaux du serveur	Recherches, filtres, paramètres qui peuvent être partagés
POST	Les données sont envoyées dans le corps de la requête (absentes de l'URL)	Formulaires de connexion, d'inscription, actions qui modifient des données

Bonnes pratiques :

- Utiliser **POST** pour les mots de passe et pour toute action qui modifie des données.
- Utiliser **GET** pour des paramètres qui peuvent être mis en favori ou partagés.

- **POST n'est pas « sécurisé » en soi** : les données ne sont pas chiffrées. Seul **HTTPS** protège la confidentialité des échanges.

Formulaire HTML – GET

L'attribut `name` de chaque champ devient la **clé** du tableau `$_GET` (ou `$_POST`).

```
<form method="get">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom">
    <input type="submit" value="Envoyer">
</form>
```

Traitement PHP

```
<?php
if (isset($_GET["nom"])) {
    echo "Bonjour " . htmlspecialchars($_GET["nom"], ENT_QUOTES, "UTF-8");
}
?>
```

Formulaire HTML – POST

```
<form method="post">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom">
    <input type="submit" value="Envoyer">
</form>
```

Traitement PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = trim($_POST["nom"] ?? "");
    echo "Bonjour " . htmlspecialchars($nom, ENT_QUOTES, "UTF-8");
}
?>
```

- `$_SERVER["REQUEST_METHOD"]` permet de savoir si la page est appelée par l'envoi du formulaire (`POST`) ou par une simple visite (`GET`).
- `??` fournit une valeur par défaut : sans lui, lire `$_POST["nom"]` alors que le champ n'a pas été envoyé produit l'avertissement `Undefined array key "nom"`.
- `trim()` retire les espaces en début et en fin de chaîne, mais **ne valide pas** la donnée.
- `htmlspecialchars($nom, ENT_QUOTES, "UTF-8")` transforme `<`, `>`, `&`, `"` et `'` en entités HTML : le texte saisi est **affiché** et non interprété comme du HTML, ce qui empêche les attaques **XSS** (injection de code dans la page).

`isset()`, `empty()` ou `??` ?

Test	Vrai quand…	Piège
`isset($x)`	la variable existe et n'est pas `null`	Un champ vide envoyé (`""`) est « défini »
`empty($x)`	la variable n'existe pas ou vaut `""`, `0`, `"0"`, `false`, `[]`, `null`	`empty("0")` est vrai : la saisie `0` serait refusée
`$x ?? "defaut"`	donne une valeur par défaut si `$x` n'existe pas / est `null`	—

> **Conseil** : lire les champs avec `trim($_POST["champ"] ?? "")`, puis comparer à `""` (`=== ""`).

Validation d'un email

```php
$email = trim($_POST["email"] ?? "");

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email invalide";
}
```

- `filter_var($valeur, FILTER_VALIDATE_EMAIL)` vérifie qu'une chaîne a la **forme** d'une adresse électronique valide (elle ne vérifie pas que la boîte existe).
- Variante : `filter_input(INPUT_POST, "email", FILTER_VALIDATE_EMAIL)` renvoie `false` si l'adresse est invalide et `null` si le champ est absent.
- Autres filtres utiles : `FILTER_VALIDATE_INT`, `FILTER_VALIDATE_FLOAT`, `FILTER_VALIDATE_IP`, `FILTER_VALIDATE_URL`.

Structure type d'un traitement de formulaire

```php
<?php
$erreurs = [];
$nom = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = trim($_POST["nom"] ?? "");

    if ($nom === "") {
        $erreurs[] = "Le nom est obligatoire.";
    }

    if (empty($erreurs)) {
        // traitement : enregistrement, message de succès...
    }
}
?>
<?php foreach ($erreurs as $erreur) : ?>
    <p style="color: red;"><?= htmlspecialchars($erreur, ENT_QUOTES, "UTF-8") ?></p>
<?php endforeach; ?>

<form method="post">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom"
           value="<?= htmlspecialchars($nom, ENT_QUOTES, "UTF-8") ?>">
    <button type="submit">Envoyer</button>
</form>
```

Idée clé : **initialiser** les variables, **tester** la méthode, **valider** chaque champ, **cumuler** les erreurs, **réafficher** la saisie (échappée).

Remarques

- Les contrôles HTML (`required`, `type="email"`, `pattern`) améliorent le confort mais **peuvent être contournés** : la validation en PHP est obligatoire.
- Éviter de mélanger GET et POST pour le même formulaire.
- Après un traitement réussi qui modifie des données, on **redirige** (`header("Location: ...")` puis `exit;`) pour éviter le renvoi du formulaire lorsque l'utilisateur recharge la page.

---

10. Inclusion de fichiers

```php
include "header.php";
require "config.php";
require_once "db.php";
```

- `include` → simple avertissement si le fichier est absent, le script continue.
- `require` → erreur fatale si le fichier est absent, le script s'arrête.
- `include_once` / `require_once` → n'incluent le fichier qu'**une seule fois** (évite les doublons, par exemple de fonctions ou de connexions).

Usage typique : `header.php` / `footer.php` (structure commune des pages), `db.php` (connexion à la base), `fonctions.php` (fonctions réutilisables).

---

11. Sessions et cookies

Le protocole HTTP est **sans mémoire** : chaque requête est indépendante. Pour reconnaître un utilisateur d'une page à l'autre (connexion, panier), on utilise une **session**.

Principe

1. `session_start()` crée une session (ou reprend celle du visiteur) et envoie au navigateur un **cookie** contenant un identifiant (`PHPSESSID`).
2. Les données sont stockées **sur le serveur** dans le tableau `$_SESSION`.
3. À chaque requête, le navigateur renvoie l'identifiant : PHP retrouve les données.

> `session_start()` doit être appelé **avant tout affichage** (avant le moindre HTML, espace ou `echo`), sinon : `headers already sent`.

Exemple : compteur de visites

```php
<?php
session_start();

$_SESSION["visites"] = ($_SESSION["visites"] ?? 0) + 1;
?>
<p>Vous avez visité cette page <?= (int) $_SESSION["visites"] ?> fois.</p>
```

Connexion, page protégée et déconnexion

`login.php` (après vérification des identifiants) :

```php
session_start();
// ... identifiants vérifiés ...
session_regenerate_id(true);           // nouvel identifiant après connexion
$_SESSION["utilisateur"] = $identifiant;
header("Location: compte.php");
exit;
```

`compte.php` (page réservée aux utilisateurs connectés) :

```php
<?php
session_start();

if (!isset($_SESSION["utilisateur"])) {
    header("Location: login.php");
    exit;
}
?>
<h1>Bienvenue <?= htmlspecialchars($_SESSION["utilisateur"], ENT_QUOTES, "UTF-8") ?></h1>
<a href="logout.php">Se déconnecter</a>
```

`logout.php` :

```php
<?php
session_start();
$_SESSION = [];                                                    // vide les
données
setcookie(session_name(), "", ["expires" => time() - 3600, "path" => "/"]);  // supprime le
cookie PHPSESSID
session_destroy();                                                 // détruit la
session côté serveur
header("Location: login.php");
exit;
```

> `session_destroy()` seule ne suffit pas : le cookie `PHPSESSID` resterait dans le navigateur.

> Après un `header("Location: ...")`, toujours écrire `exit;` : sinon le reste du script continue à s'exécuter.

Cookies

Un **cookie** est une petite donnée stockée **dans le navigateur du client** (préférence de thème, langue…).

```php
// Création (avant tout affichage) : 30 jours, tout le site
setcookie("theme", "sombre", [
    "expires"  => time() + 3600 * 24 * 30,
    "path"     => "/",
    "httponly" => true,      // invisible pour JavaScript
    "samesite" => "Lax",     // limite l'envoi depuis d'autres sites
]);

// Lecture : disponible à la requête suivante, valeur envoyée par le client → à échapper
$theme = $_COOKIE["theme"] ?? "clair";
echo htmlspecialchars($theme, ENT_QUOTES, "UTF-8");
```

	Session (`$_SESSION`)	**Cookie (`$_COOKIE`)**
Stockage	Serveur (le client n'a que l'identifiant)	Navigateur du client
Contrôle par l'utilisateur	Non	Oui : il peut le lire et le modifier
Usage	Connexion, panier, données sensibles	Préférences non sensibles

> Ne **jamais** faire confiance à un cookie ni y stocker de mot de passe ou de donnée sensible.

---

12. PHP et base de données MariaDB avec PDO

PDO (*PHP Data Objects*) est l'interface standard pour interroger une base de données depuis PHP.

Préparer la base (phpMyAdmin : `http://localhost/phpmyadmin`)

```
CREATE DATABASE boutique CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE boutique;

CREATE TABLE utilisateurs (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    identifiant VARCHAR(50) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    cree_le DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE produits (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prix DECIMAL(8,2) NOT NULL,
    stock INT UNSIGNED NOT NULL DEFAULT 0
);
```

> La colonne `mot_de_passe` fait 255 caractères : elle stockera un **hash**, jamais le mot de passe en clair.

Connexion : `db.php`

```
<?php
$dsn = "mysql:host=localhost;dbname=boutique;charset=utf8mb4";

try {
    $pdo = new PDO($dsn, "root", "", [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    exit("Connexion à la base impossible.");
}
```

> `root` sans mot de passe est la configuration **par défaut de XAMPP, valable uniquement en local**. En production : utilisateur dédié, avec uniquement les droits nécessaires, et mot de passe fort.

Lire des données : `SELECT`

```
require_once "db.php";

$produits = $pdo->query("SELECT id, nom, prix FROM produits ORDER BY nom")->fetchAll();

foreach ($produits as $p) {
    echo htmlspecialchars($p["nom"], ENT_QUOTES, "UTF-8") . " : "
       . number_format($p["prix"], 2, ",", " ") . " €<br>";
}
```

Requêtes préparées : le réflexe indispensable

Ne jamais insérer une donnée venant de l'utilisateur directement dans une requête SQL :

```php
// DANGEREUX : injection SQL
$sql = "SELECT * FROM utilisateurs WHERE identifiant = '" . $_POST["identifiant"] . "'";
// Saisie : ' OR '1'='1   → la condition devient toujours vraie
```

La **requête préparée** envoie séparément la requête et les valeurs : la donnée ne peut plus modifier la structure de la requête.

```php
$stmt = $pdo->prepare("SELECT id, mot_de_passe FROM utilisateurs WHERE identifiant
= :identifiant");
$stmt->execute(["identifiant" => $identifiant]);
$utilisateur = $stmt->fetch();     // tableau associatif, ou false si aucun résultat
```

Écrire des données : `INSERT`

```php
$stmt = $pdo->prepare("INSERT INTO utilisateurs (identifiant, mot_de_passe) VALUES
(:id, :mdp)");

try {
    $stmt->execute([
        "id"  => $identifiant,
        "mdp" => password_hash($motDePasse, PASSWORD_DEFAULT),
    ]);
    $message = "Compte créé.";
} catch (PDOException $e) {
    if ($e->getCode() === "23000") {          // violation de contrainte UNIQUE
        $erreurs[] = "Cet identifiant existe déjà.";
    } else {
        throw $e;
    }
}
```

Méthode	Rôle
`$pdo->query($sql)`	Exécute une requête **sans paramètre**
`$pdo->prepare($sql)`	Prépare une requête avec paramètres (`:nom` ou `?`)
`$stmt->execute([...])`	Exécute avec les valeurs
`$stmt->fetch()`	Lit **une** ligne
`$stmt->fetchAll()`	Lit **toutes** les lignes
`$stmt->rowCount()`	Nombre de lignes affectées (`UPDATE` , `DELETE`)

13. Bonnes pratiques de sécurité

Risque	Principe	En PHP
Données non fiables	Toujours **valider** côté serveur	`trim()`, `filter_var()`, `preg_match()`, longueurs
XSS (injection de HTML/JS)	**Échapper** à l'affichage	`htmlspecialchars($x, ENT_QUOTES, "UTF-8")`
Injection SQL	Ne jamais concaténer une saisie dans le SQL	Requêtes préparées PDO
Mots de passe volés	Ne jamais stocker en clair	`password_hash()` / `password_verify()`
Vol / fixation de session	Renouveler l'identifiant à la connexion	`session_regenerate_id(true)`
Écoute du réseau	Chiffrer les échanges	HTTPS (surtout pour les formulaires de connexion)
Fuite d'informations	Ne pas afficher les erreurs en production	`display_errors = Off`, journaliser les erreurs

Hacher et vérifier un mot de passe

```php
// À l'inscription
$hash = password_hash($motDePasse, PASSWORD_DEFAULT);   // à stocker en base

// À la connexion
if ($utilisateur && password_verify($motDePasse, $utilisateur["mot_de_passe"])) {
    // connexion réussie
}
```

- Un **hash** est à sens unique : on ne peut pas retrouver le mot de passe, on ne peut que vérifier une saisie.
- `password_hash()` ajoute automatiquement un **sel** aléatoire : deux mots de passe identiques donnent deux hashs différents. Ne jamais utiliser `md5()` ni `sha1()` pour les mots de passe.
- Message d'erreur **générique** à la connexion (« Identifiant ou mot de passe incorrect ») : ne pas révéler lequel des deux est faux.

Protection contre le CSRF

Une attaque **CSRF** pousse le navigateur d'un utilisateur connecté à envoyer, à son insu, une requête vers votre site. Parade : un jeton secret dans chaque formulaire.

À l'affichage du formulaire (la session doit être démarrée) :

```php
<?php
session_start();
$_SESSION["csrf"] ??= bin2hex(random_bytes(32));   // un jeton par session
?>
<form method="post">
    <input type="hidden" name="csrf"
           value="<?= htmlspecialchars($_SESSION["csrf"], ENT_QUOTES, "UTF-8") ?>">
    <!-- … autres champs … -->
    <button type="submit">Envoyer</button>
</form>
```

Au traitement :

```php
<?php
session_start();

if (!hash_equals($_SESSION["csrf"] ?? "", $_POST["csrf"] ?? "")) {
    http_response_code(403);
    exit("Requête refusée.");
}
```

> Attention : dans un commentaire sur une ligne (`//` ou `#`), la séquence `?>` **termine le bloc PHP**. Ne jamais y écrire de code PHP « en exemple ».

Configuration des erreurs

```php
// Développement
ini_set("display_errors", "1");
error_reporting(E_ALL);

// Production : display_errors à 0, erreurs écrites dans un fichier de log
```

---

14. Erreurs fréquentes et débogage

Symptôme	Cause probable	Solution
Le code PHP s'affiche tel quel dans la page	Fichier ouvert en `file://` ou extension `.html`	Passer par `http://localhost/...`, extension `.php`
Page blanche	Erreur fatale masquée	Activer `display_errors` et `E_ALL` en développement
`Parse error: syntax error, unexpected ...`	`;`, `)`, `}` ou guillemet oublié	Regarder la ligne indiquée **et la précédente**
`Undefined variable $x` / `Undefined array key "nom"`	Variable ou champ jamais défini	Initialiser la variable, utiliser `??`
`Cannot modify header information - headers already sent`	Affichage avant `header()` ou `session_start()`	Mettre ces appels tout en haut, sans espace avant `<?php`
`Call to undefined function ...`	Faute de frappe ou extension PHP non activée	Vérifier le nom, `phpinfo()`
`SQLSTATE[HY000] [1045] Access denied`	Mauvais identifiants de base de données	Vérifier utilisateur / mot de passe dans `db.php`
`SQLSTATE[HY000] [2002] Connection refused`	MariaDB n'est pas démarré	Lancer le module MySQL dans XAMPP
Accents illisibles (`Ã©`)	Mauvais encodage	Fichier en UTF-8, `<meta charset="UTF-8">`, `charset=utf8mb4` dans le DSN

Méthode de débogage :

1. Lire **entièrement** le message d'erreur (type, fichier, numéro de ligne).
2. Afficher les valeurs avec `var_dump()` entre `<pre>` … `</pre>`.
3. Isoler : réduire le code jusqu'à trouver l'instruction fautive.
4. Tester **une modification à la fois**.
